

CH05 Spring Boot

考试复习要点总结 (Exam Review Summary)

软件架构平台 | 计算机学院

基于课堂课件整理，页码引用格式 [p.X] 对应课件页码

1 Spring Boot 概述 (Overview of Spring Boot)

1.1 为什么需要 Spring Boot? (Why Spring Boot?)

- 传统 Spring 框架的痛点 (Pain Points of Traditional Spring) [p.5]
 - 几乎所有 Spring 组件或技术都基于 **Spring IoC** (控制反转) 和 **Spring AOP** (面向切面编程)。[p.5]
 - 每个组件或技术又还有自身的相关配置。[p.5]
 - Web 容器和数据库等还有一些其他的相关配置。[p.5]
 - 导致在使用 Spring 框架相关技术时，“搭环境”往往比“写代码”更耗时、更容易出错。[p.5]
- 传统 Spring Web 应用开发流程 (Traditional Development Flow) [p.8]
 - 配置环境 → 创建工程 → 构建目录结构 → 组件依赖管理 → 设置组件参数 → 配置 Web 容器 → 业务开发 → 测试与构建 → 部署与发布 → 运维与监控。[p.8]
 - 其中大量工作是重复性的配置，严重降低了开发效率。[p.6]

1.2 什么是 Spring Boot? (What is Spring Boot?)

- 基本概念 (Core Concept) [p.6]
 - Spring Boot 是 Spring 为简化 Spring 框架的使用，推出的一个 组件/工具。[p.6]
 - Spring Boot 是一个基于 Spring 的 快速开发脚手架 (Scaffold)，其核心设计哲学是 “约定优于配置” (Convention over Configuration)。[p.6]
 - Spring Boot 不是对 Spring 框架功能上的替代，而是对 Spring 使用方式的简化。[p.6]
 - Spring Boot 本身也不是类似于 Spring MVC 具有某种功能的 Spring 组件。[p.6]
- 版本信息 (Version Info) [p.7]
 - 官网: <https://spring.io/projects/spring-boot/>。[p.7]
 - 目前最新稳定版为 4.0.3 (Spring 7.x、JDK 17+)。[p.7]
- 核心目标 (Core Goals) [p.7]
 1. 极低的学习成本和极大的提高开发效率。[p.7]
 2. 开发可独立运行的 Web 应用。[p.7]
 3. 简单的组件依赖，自动发现与自动装配 (Auto-Discovery & Auto-Wiring)。[p.7]
 4. 提供运行时的应用监控 (Runtime Monitoring)。[p.7]
 5. 提供与分布式架构、云原生 (Cloud-Native) 架构和大数据等组件的良好集成。[p.7]

1.3 脚手架的解决思路 (Scaffold Approach)

- Spring Boot 对传统流程的简化 [p.9]
 - 配置环境、创建工程、构建目录结构 → 由 **Spring Initializr** 一键完成，简化操作。[p.9]
 - 设置组件参数、配置 Web 容器、组件依赖管理 → 由 Spring Boot 自动配置机制完成。[p.9]
 - 业务开发、测试与构建、部署与发布 → 开发人员专注业务逻辑。[p.9]
 - 运维与监控 → 通过 **Spring Actuator** 工具提供。[p.9]

2 Spring Boot 快速入门 (Quick Start)

2.1 利用 Spring Initializr 快速构建应用

- 使用 IntelliJ IDEA 内置的 Spring Initializr 向导快速构建 Spring Boot 应用。 [p.10]
- 选择项目所需的组件（如 Web、JPA、Security 等 Starter），自动生成项目骨架 (Project Skeleton)。 [p.11]
- 创建方式 (Creation Methods) [p.10–11]
 - IDE 集成方式：IDEA Ultimate 内置 Spring Initializr 向导。 [p.10]
 - Web 方式：访问 <https://start.spring.io> 在线生成并下载项目。 [p.10]

2.2 项目目录结构 (Project Directory Structure)

- Spring Boot 约定的 Web 项目目录结构特点 [p.12]:
 - 直接运行的 main 方法。 [p.12]
 - 集成所有配置的默认 application.properties。 [p.12]
 - 符合 Maven 标准目录布局。 [p.12–13]

2.2.1 标准目录布局 (Standard Directory Layout) [p.13]

目录/文件	说明
/src/main/java	Java 源代码目录（含启动类与业务代码）。 [p.13]
/src/main/resources	资源目录（配置文件、静态资源等）。 [p.13]
/src/main/resources/static	静态资源目录（CSS、JS、图片等）。 [p.13]
/src/main/resources/templates	表示层页面目录（Thymeleaf / JSP 模板）。 [p.13]
/src/main/resources/application.properties	Spring Boot 核心配置文件，集成所有配置。 [p.13]
/src/test	测试文件目录（单元测试与集成测试）。 [p.13]

2.3 运行项目 (Run the Project)

- 直接运行 main 方法启动 Spring Boot 应用，无需部署到外部服务器。 [p.14]
- 启动类示例：带有 @SpringBootApplication 注解的类，包含 public static void main 方法。 [p.12–14]

3 Spring Boot 自动配置原理 (Auto-Configuration Principles)

3.1 四大组成 (Four Pillars of Automation)

[p.16]

Spring Boot 的自动化和“开箱即用”(Out-of-the-Box) 功能主要由以下四方面组成：

1. 依赖管理的 Starter 机制：基于 Maven 提供简化和统一的依赖管理。 [p.16]
2. Spring IoC 自动配置 Bean 机制：大量条件判断 + 大量默认值，自动装配 Bean。 [p.16]
3. 统一集成的配置文件：application.properties 或 application.yml。 [p.16]
4. 内嵌式 Servlet 容器：嵌入式 Tomcat，无需外部服务器，直接 java -jar 运行。 [p.16]

3.2 Starter 机制 (Starter Mechanism)

3.2.1 Starter 概念与原理 (Concept and Principle)

- Starter 定义 [p.17]：Spring Boot 基于 Maven 提供简化和统一的依赖管理。Starter 将某一功能领域所需的所有依赖打包在一起，确保版本兼容。 [p.17]

- **spring-boot-starter-parent** [p.17]: 每个项目都可以继承的父 POM (Project Object Model), 其中定义了各种常用依赖的版本和关系, 确保了项目中各种第三方依赖的兼容性和依赖关系。[p.17]
- **版本仲裁 (Version Arbitration)**: 继承 parent POM 后, 引入官方 Starter 时可以不写版本号, 由父 POM 统一管理版本。[p.17]
- **命名规范 (Naming Convention)** [p.17–18]:
 - 官方 Starter: `spring-boot-starter-*` 格式 (如 `spring-boot-starter-web`)。[p.17]
 - 第三方 Starter: `*-spring-boot-starter` 格式。[p.18]

3.2.2 常用官方 Starter 对比表 (Official Starter Comparison Table)

Starter	核心依赖	说明
spring-boot-starter-web	Spring MVC, Jackson, Validation, 内嵌 Tomcat	构建 Web 应用 (含 RESTful), 包含 MVC 框架和嵌入式 Servlet 容器。[p.19]
spring-boot-starter-data-jpa	Spring Data JPA, Hibernate, 数据库驱动	集成 JPA (Java Persistence API), 提供 JpaRepository 接口, 简化数据访问层。[p.19]
spring-boot-starter-security	Spring Security	集成安全框架, 提供默认登录界面与安全配置, 实现认证 (Authentication) 和授权 (Authorization)。[p.19]
spring-boot-starter-test	JUnit, Mockito, AssertJ, Spring Test	测试 Starter, 集成单元测试和集成测试所需库。[p.19]
spring-boot-starter-thymeleaf	Thymeleaf 模板引擎与解析器	集成 Thymeleaf 模板引擎, 作为服务端渲染方案。[p.19]
spring-boot-starter-aop	Spring AOP 自动代理	启用 Spring AOP 面向切面编程支持 (如日志切面、事务切面)。[p.19]
spring-boot-starter-actuator	Actuator 监控端点	提供生产就绪的监控端点 (Endpoints) 和指标收集 (Metrics)。[p.19]
spring-boot-starter-logging	SLF4J + Logback	Spring Boot 默认日志方案, 通过 SLF4J 门面 + Logback 实现。[p.27]
spring-boot-starter-validation	Hibernate Validator	集成 Bean Validation (JSR-380), 支持数据校验。[p.19]

3.2.3 自动装配小结图 (Auto-Wiring Overview)

[p.27]

Spring Boot 启动装配流程:

1. 加载配置文件: 读取 `application.properties` 或 `application.yml`。[p.27]
2. 自动装配 Starter: `spring-boot-starter-web` (Web 支持)、`spring-boot-starter-data` (数据库支持)、`spring-boot-starter-logging` (日志支持) 等。[p.27]
3. 加载组件: 自动扫描并注册 `@Repository`、`@Controller`、`@Service`、`@Entity` 等注解标记的 Bean。[p.27]
4. 应用初始化: 启动内嵌容器, 注册 `DispatcherServlet`, 应用就绪。[p.27]

3.3 自动配置机制 (Auto-Configuration Mechanism)

3.3.1 自动配置工作流程 (Auto-Configuration Workflow)

- 自动配置定义 [p.20]: 自动配置是 Spring Boot 自动化中的**核心机制**。其基本原理是：**大量的条件判断 + 大量的默认值**。[p.20]
- 核心入口注解: `@SpringBootApplication` 是一个组合注解。[p.20]

自动配置四步工作流程 [p.20]:

- 步骤 1: 启动 (Start): 从 `@SpringBootApplication` 注解触发 `@EnableAutoConfiguration` 自动配置入口。[p.20]
- 步骤 2: 加载配置 (Load Configurations): 扫描所有 jar 包下的 `META-INF/spring/XXX.AutoConfiguration.imports` 文件。注意: Spring Boot 3.x 之前使用 `spring.factories`, 3.x 开始改用 `.imports` 文件格式。[p.20]
- 步骤 3: 条件评估 (Condition Evaluation): 读取该文件中列出的所有自动配置类(如 `DispatcherServletAutoConfiguration`、`DataSourceAutoConfiguration` 等)。[p.20]
- 步骤 4: 条件注解判断 (Conditional Annotation Check): 使用 `@ConditionalOnClass`、`@ConditionalOnMissingBean` 等条件注解进行条件判断。例如: 如果 classpath 下有 `DispatcherServlet.class`, 且用户没有自定义 `DispatcherServlet` Bean, 则自动配置一个 `DispatcherServlet`。[p.20]

3.3.2 @SpringBootApplication 组合注解详解

- `@SpringBootApplication` 等价于以下三个注解的组合 [p.20]:
 1. `@SpringBootConfiguration`: 本质上是一个 `@Configuration`, 标记该类为配置类, 允许在其中定义 Bean。[p.20]
 2. `@EnableAutoConfiguration`: 开启自动配置, 是 Spring Boot 自动配置机制的入口注解, 触发自动配置类加载。[p.20]
 3. `@ComponentScan`: 开启组件扫描, 默认扫描主启动类所在包及其子包下的组件(`@Component`、`@Controller`、`@Service`、`@Repository` 等)。[p.20]

3.3.3 源码剖析: DispatcherServletAutoConfiguration

以 `DispatcherServletAutoConfiguration` 为例, 深入理解自动配置原理 [p.21]:

1. `@AutoConfiguration`: 标记这是一个自动配置类 (Spring Boot 3.x 新增注解, 替代旧的 `@Configuration` 在自动配置场景下的语义)。[p.21]
2. `@ConditionalOnClass(DispatcherServlet.class)`: 只有 classpath 中存在 `DispatcherServlet` 类时该配置才生效 (通常由引用 Web Starter 决定)。[p.21]
3. `@ConditionalOnWebApplication`: 确认当前应用是一个 Web 应用 (而非命令行或批处理应用)。[p.21]
4. 内部定义 `@Bean` 方法创建:
 - `DispatcherServlet` —— Spring MVC 的核心前端控制器。[p.21]
 - `DispatcherServletRegistrationBean` —— 负责向 Servlet 容器注册 `DispatcherServlet`, 使其可以处理 HTTP 请求。[p.21]
5. `@ConditionalOnMissingBean(DispatcherServlet.class)`: 如果用户自己定义了一个 `DispatcherServlet` Bean, 则 Spring Boot 的自动配置就会失效 (**Back off**), 以用户自定义的为准。这就是“用户优先”策略的实现原理。[p.21]

3.3.4 自动配置机制描述 (Auto-Config Mechanism Description)

自动配置的核心思想可以总结为:

- 条件装配 (Conditional Assembly): 通过一系列 `@ConditionalOnXxx` 注解, 在特定条件满足时才创建对应的 Bean。[p.20-21]
- 默认值策略 (Default Value Strategy): 当用户没有自定义时, Spring Boot 提供一套合理的默认 Bean

- (如默认的 DataSource、默认的 DispatcherServlet)。[p.20–21]
- **用户优先 (User First):** 通过 @ConditionalOnMissingBean 实现 Back off 机制，一旦检测到用户自行定义了某个 Bean，自动配置即失效，以用户的为准。[p.21]
 - **按需加载 (On-Demand Loading):** 不是一次性加载所有配置类，而是根据 classpath 中的实际依赖来决定哪些自动配置类需要被评估和加载。[p.20]

3.4 条件注解参考表 (Conditional Annotation Reference Table)

注解 (Annotation)	说明 (Description)
@SpringBootApplication	组合注解 = @SpringBootConfiguration + @EnableAutoConfiguration + @ComponentScan。Spring Boot 应用入口标记。[p.20]
@EnableAutoConfiguration	启用 Spring Boot 自动配置机制，触发所有自动配置类的条件评估和加载。[p.20]
@AutoConfiguration	标记一个类为自动配置类 (Spring Boot 3.x 新增，在.imports 文件中声明)。[p.21]
@ConditionalOnClass	classpath 中存在指定类时，当前配置才生效。如 @ConditionalOnClass(DispatcherServlet.class)。[p.20–21]
@ConditionalOnMissingBean	当 Spring 容器中不存在指定 Bean 时，当前配置才生效。实现“用户优先”的 Back off 机制。 关键考点。 [p.21]
@ConditionalOnWebApplication	当应用为 Web 应用 (Servlet 环境) 时，当前配置才生效。[p.21]
@ConditionalOnProperty	当指定配置属性存在且满足条件时，当前配置才生效。常用于功能开关。[p.20]
@ConditionalOnMissingClass	classpath 中不存在指定类时，当前配置才生效。[p.20]
@ConditionalOnBean	Spring 容器中存在指定 Bean 时，当前配置才生效。[p.20]
@ConditionalOnResource	classpath 下存在指定资源文件时，当前配置才生效。[p.20]
@ConditionalOnExpression	基于 SpEL 表达式结果决定是否生效。[p.20]
@ConditionalOnJava	Java 版本满足指定范围时，当前配置才生效。[p.20]
@ConditionalOnNotWebApplication	当前应用不是 Web 应用时，配置才生效。[p.20]
@ComponentScan	组件扫描，自动发现并注册 @Component, @Service, @Repository, @Controller 等。[p.20]
@SpringBootConfiguration	等价于 @Configuration，标记配置类 (Spring Boot 特化版本)。[p.20]

3.5 配置文件 (Configuration Files)

3.5.1 配置文件类型与基本用法

- 配置文件类型 [p.22]:

- `application.properties`: 键值对格式, 传统 Java Properties 文件。[p.22]
- `application.yml` (或 `.yaml`): YAML 格式, 层级化结构, 更易读, **推荐使用**。[p.22]
- 配置文件位置 [p.13]: 默认位于 `src/main/resources/` 目录下。
- **Java Config 方式** [p.23]: 将配置封装成配置类(`@Configuration`), 通过 `@Bean` 或 `@ConfigurationProperties` 进行属性绑定。[p.23]
 - `@Value`: 注入单个属性值, 支持 SpEL 表达式。[p.23]
 - `@ConfigurationProperties`: 批量映射一组相关配置到 POJO 类, 实现**类型安全**的配置管理。[p.23]
- 配置文件加载优先级 (从高到低) [p.22-23]: 命令行参数 → 环境变量 → 外部 `application.yml` → 内部 (classpath) `application.yml`。

3.5.2 多环境配置与 @Profile (Multi-Environment Configuration)

- 需求场景 [p.24]: 开发环境 (dev)、测试环境 (test)、生产环境 (prod) 配置不同 (如端口号、数据库连接、日志级别等)。[p.24]
- 配置文件命名规范 [p.24]:
 - 格式: `application-{profile}.yml`。
 - 示例: `application-dev.yml`、`application-test.yml`、`application-prod.yml`。[p.24]
 - 通用配置放在 `application.yml` 中, 环境特有配置放在 `application-{profile}.yml` 中。[p.24]
- 激活方式 (Activation Methods) [p.24]:
 1. 配置文件指定: 在 `application.yml` 中设置 `spring.profiles.active: dev`。[p.24]
 2. 命令行参数激活: `java -jar myapp.jar --spring.profiles.active=prod`。[p.24]
 3. 环境变量激活: 设置 `SPRING_PROFILES_ACTIVE=prod`。[p.24]
- **@Profile** 注解 [p.24]: 可以用在 `@Component`、`@Configuration` 或 `@Bean` 上, 指定该组件/配置仅在特定环境下生效 (如 `@Profile("dev")` 仅在开发环境生效)。[p.24]

3.6 内嵌 Servlet 容器 (Embedded Servlet Container)

- 概念 [p.25]: Spring Boot 默认使用**嵌入式 Tomcat** 作为 Servlet 容器, 还支持 Jetty、Undertow 等作为替代。[p.25]
- 核心特性 [p.25]: Web 应用无需部署到外部服务器, 直接 `java -jar` 即可运行。[p.25]

3.6.1 传统模式 vs Spring Boot 模式 (Traditional vs Spring Boot)

- 传统模式 [p.25]: 开发应用 (WAR 包) → 丢进外部 Tomcat → Tomcat 启动 → 加载应用。开发、部署流程复杂。[p.25]
- **Spring Boot 模式** [p.25]:

main 方法 → `SpringApplication.run()` → 创建 `ApplicationContext` → 启动内嵌 Tomcat / Jetty → 注册 `DispatcherServlet` → 应用就绪

- Spring Boot 通过 Java 代码在运行时**动态创建 Tomcat 实例**, 并将当前应用注册进去, 因此可以直接通过 `java -jar` 运行。[p.25-26]
- 容器切换 [p.25]: 在 `pom.xml` 中排除 `spring-boot-starter-tomcat` 依赖, 并引入 Jetty 或 Undertow 对应的 Starter 即可切换。[p.25]

3.6.2 Servlet 容器常用配置 (Container Configuration)

[p.26]

配置项	说明	示例
server.port	修改 HTTP 端口，默认 8080。也可通过环境变量 SERVER_PORT 覆盖。 [p.26]	server.port=9090
server.port=-1	创建 WebApplicationContext 但不打开端口（关闭 HTTP 服务）。 [p.26]	server.port=-1
server.port=0	随机端口，让系统自动选取一个可用端口。 [p.26]	server.port=0
server.compression.enabled	启用 HTTP 响应压缩 (Gzip)。 [p.26]	true
server.ssl.key-store	SSL 证书存储路径。 [p.26]	classpath:keystore.p12
server.ssl.key-store-password	SSL 证书密码。 [p.26]	secret
server.servlet.context-path	应用上下文路径。 [p.26]	/api
server.tomcat.max-threads	Tomcat 最大工作线程数。 [p.26]	200

4 项目中使用范例 (Usage Examples in Projects)

4.1 日志管理 (Logging Management)

4.1.1 日志基础概念 (Logging Fundamentals)

- 日志的作用 [p.29]: 在程序运行过程中，为监测某些功能或验证某些指标是否正确，需要输出相关信息。`System.out.println()` 是最简单的日志处理方式，但缺点明显：无法控制级别、性能差、无法持久化。 [p.29]
- JUL (`java.util.logging`) [p.29]: JDK 内置的日志框架，提供了基本的日志功能（Logger、Handler、Formatter）。 [p.29]

4.1.2 日志门面与日志实现 (Logging Facade and Implementation)

[p.30]

- 日志门面 (Logging Facade) [p.30]: 只提供日志相关的接口定义（API），而不提供具体的接口实现。日志门面解除了接口和实现的耦合，使用户可以灵活地选择日志的具体实现框架。 [p.30]
 - 常见日志门面: Commons-Logging (JCL)、SLF4J (Simple Logging Facade for Java)。 [p.30]
- 日志实现 (Logging Implementation) [p.30]: 与日志门面对应，提供了具体的日志接口实现，应用程序通过它执行日志打印的功能。 [p.30]
 - 常见日志实现: Log4j、JUL (`java.util.logging`)、Logback、Log4j2。 [p.30]
- 设计模式 [p.30]: 门面模式 (Facade Pattern) —— 面向接口编程，解耦应用代码与具体日志实现。应用程序通过日志门面 API 写日志，底层可以随时替换日志实现框架，只需更换依赖，无需修改代码。 [p.30]

4.1.3 Spring Boot 中的日志管理 (Logging in Spring Boot)

[p.32]

- 默认日志方案 [p.32]: Spring Boot 默认使用 **SLF4J** (门面) + **Logback** (实现) 作为日志解决方案。 [p.32]
- 日志级别 (Log Levels) (从低到高) [p.32]:

TRACE → DEBUG → INFO → WARN → ERROR → FATAL → OFF

- 通过配置文件自定义日志 [p.32]:
 - `logging.level.root=INFO`: 设置根日志级别。[p.32]
 - `logging.level.org.springframework=DEBUG`: 按包名细化日志级别。[p.32]
 - `logging.file.name=app.log`: 指定日志输出文件名称。[p.32]
 - `logging.file.path=/var/log`: 指定日志输出目录。[p.32]
 - `logging.pattern.console`: 自定义控制台输出格式。[p.32]
 - `logging.pattern.file`: 自定义文件输出格式。[p.32]
- 日志框架切换 [p.32]: 排除默认 Logback 依赖, 引入 Log4j2 对应的 Starter, 即可切换为 Log4j2。[p.32]
- 打印日志的方式 (在代码中使用 Lombok 简化):
 - 使用 `@Slf4j` 注解 (Lombok), 在类中直接调用 `log.info()`、`log.error()` 等。[p.32]
 - 或手动创建 Logger: `LoggerFactory.getLogger(ClassName.class)`。[p.32]

4.2 事务管理 (Transaction Management)

4.2.1 事务的 ACID 属性 (ACID Properties of Transactions)

[p.33]

场景 [p.33]: A 账户向 B 账户转账 100 元。

- 步骤 1: A 账户扣款 100 元。[p.33]
- 步骤 2: B 账户加款 100 元。[p.33]
- 问题: 步骤 1 成功, 步骤 2 失败 (网络中断、服务器宕机), 钱就“消失”了。[p.33]
- 解决方案: 要么全部成功, 要么全部失败——这就是**事务 (Transaction)**。[p.33]

ACID 四大属性 [p.33]:

1. **原子性 (Atomicity)**: 事务中的所有操作作为一个**整体**, 不可分割。任何一个操作失败, 整个事务回滚 (Rollback)。[p.33]
2. **一致性 (Consistency)**: 事务执行前后, 数据完整性约束没有被破坏。例如转账前后总金额不变。[p.33]
3. **隔离性 (Isolation)**: 多个事务并发执行时, 彼此互不干扰。每个事务看到的数据视图只取决于其本身, 防止脏读、不可重复读、幻读等问题。[p.33]
4. **持久性 (Durability)**: 事务一旦提交 (Commit), 对数据的修改是**永久性的**, 即使系统崩溃也不会丢失。[p.33]

4.2.2 编程式事务 (Programmatic Transaction)

[p.34]

- 实现方式 [p.34]: Spring 中提供 `TransactionTemplate` 或 `PlatformTransactionManager` 手动控制事务的提交和回滚。[p.34]
- 特点 [p.34]:
 - 优点: 灵活, 可细粒度控制事务边界 (如部分提交、嵌套事务等特殊场景)。[p.34]
 - 缺点: 代码冗余, 与业务逻辑耦合, 侵入性强, 维护成本高。[p.34]
- 适用场景 [p.36]: 需要精细控制事务边界的小范围场景, 或无法使用 AOP 代理的特定情况。[p.36]

4.2.3 声明式事务与 @Transactional (Declarative Transaction)

[p.35]

- 实现方式 [p.35]: Spring 中基于 **AOP (面向切面编程)**, 通过 `@Transactional` 注解实现声明式事务。[p.35]
- 工作原理 [p.35]: Spring AOP 在运行时生成代理对象, 在 `@Transactional` 标记的方法调用前后自动开启/提交/回滚事务。[p.35]

- 特点 [p.35]:
 - 优点: 业务代码干净, 无侵入, 与实际业务逻辑解耦。实际项目开发中大部分场景使用声明式事务。 [p.35]
 - 注意事项: `@Transactional` 默认只对 `RuntimeException` 和 `Error` 回滚 (非受检异常), 对 `CheckedException` 不自动回滚。 [p.35]
- `@Transactional` 主要属性 [p.35]:
 - `propagation`: 事务传播行为 (`REQUIRED`、`REQUIRES_NEW`、`NESTED`、`SUPPORTS` 等)。 [p.35]
 - `isolation`: 事务隔离级别 (`DEFAULT`、`READ_UNCOMMITTED`、`READ_COMMITTED`、`REPEATABLE_READ`、`SERIALIZABLE`)。 [p.35]
 - `timeout`: 事务超时时间 (单位: 秒)。 [p.35]
 - `readOnly`: 是否只读事务 (默认 `false`, 设为 `true` 可做性能优化, 但不能修改数据)。 [p.35]
 - `rollbackFor` / `noRollbackFor`: 指定回滚/不回滚的异常类型。 [p.35]

4.2.4 编程式事务 vs 声明式事务对比 (Comparison)

[p.34–36]

对比维度	编程式事务 (Programmatic)	声明式事务 (Declarative)
实现方式	通过 <code>TransactionTemplate</code> 或 <code>PlatformTransactionManager</code> 手动编码。 [p.34]	通过 <code>@Transactional</code> 注解, 基于 AOP 自动代理。 [p.35]
代码侵入性	高, 事务代码与业务逻辑混合。 [p.34]	低, 业务代码干净, 无侵入。 [p.35]
灵活性	高, 可细粒度控制事务边界。 [p.34]	中等, 适合大多数标准事务场景。 [p.35]
维护成本	高, 代码冗余。 [p.34]	低, 注解方式简洁。 [p.35]
使用场景	部分提交、嵌套事务、精确控制事务边界的特殊场景。 [p.36]	常规 CRUD 操作、Service 层方法级别事务控制 (项目主流)。 [p.35]
使用频率	少。 [p.36]	高, 绝大部分场景。 [p.35]

4.3 Spring Actuator 运维监控 (Actuator Monitoring)

- **Actuator 定义** [p.9, p.19]: Spring Actuator 是 Spring Boot 提供的**运维监控工具**, 提供生产就绪 (Production-Ready) 的功能, 帮助监控和管理应用。 [p.9]
- **引入方式** [p.19]: 添加 `spring-boot-starter-actuator` Starter 依赖即可启用。 [p.19]
- **暴露方式** [p.19]: 监控端点 (Endpoints) 可通过 HTTP 或 JMX 方式暴露。 [p.19]

4.3.1 常用 Actuator 端点 (Common Endpoints)

[p.19]

端点 (Endpoint)	说明 (Description)
---------------	------------------

/actuator/health	健康检查，返回应用运行状态（UP / DOWN），可与监控系统集成。[p.19]
/actuator/info	应用基本信息展示（可在 application.properties 中自定义 info.* 属性）。[p.19]
/actuator/metrics	应用运行指标：JVM 内存使用、线程数、HTTP 请求数、GC 次数等。[p.19]
/actuator/env	应用环境属性：所有配置文件属性、环境变量、系统属性等。[p.19]
/actuator/beans	Spring IoC 容器中所有的 Bean 列表及其依赖关系。[p.19]
/actuator/loggers	查看和运行时修改日志级别（无需重启应用）。[p.19]
/actuator/mappings	所有 URL 请求映射信息（Controller 映射、Filter 等）。[p.19]
/actuator/threaddump	线程转储 (Thread Dump)，用于诊断死锁和线程问题。[p.19]
/actuator/heapdump	堆转储 (Heap Dump)，用于分析内存泄漏。[p.19]

5 本章小结 (Chapter Summary)

[p.37]

5.1 Spring Boot 概述

- 为什么需要 Spring Boot? [p.5, p.37]: 传统 Spring 框架组件配置复杂，“搭环境”比“写代码”更耗时易错。[p.5, p.37]
- Spring Boot 是什么? [p.6, p.37]: 基于 Spring 的快速开发脚手架，核心设计哲学是“约定优于配置 (Convention over Configuration)”。不是对 Spring 功能的替代，而是使用方式的简化。[p.6, p.37]
- 核心目标 [p.7, p.37]: 极低学习成本、提高开发效率、独立运行、自动装配、运行时监控、云原生集成。[p.7, p.37]

5.2 Spring Boot 自动配置原理

- Starter 机制 [p.17, p.37]: 基于 Maven 的统一依赖管理，通过继承 spring-boot-starter-parent 实现版本仲裁，确保各种第三方依赖的兼容性。[p.17, p.37]
- 自动配置 [p.20, p.37]: 基本原理——“大量的条件判断 + 大量的默认值”。从 @SpringBootApplication → @EnableAutoConfiguration → 扫描 .imports 文件 → 条件注解判断 → 自动装配 Bean。通过 @ConditionalOnMissingClass 实现 Back off（用户优先）机制。[p.20–21, p.37]
- 配置文件 [p.22–24, p.37]: application.properties 或推荐 application.yml。支持 Profile 多环境配置（dev/test/prod），通过 spring.profiles.active 或命令行参数激活。[p.22–24, p.37]
- 内嵌 Servlet 容器 [p.25, p.37]: 默认嵌入式 Tomcat，通过 Java 代码在运行时动态创建，无需部署到外部服务器，直接 java -jar 运行。[p.25, p.37]

5.3 项目中使用范例

- **日志管理** [p.29–32, p.37]: 日志门面 (Facade) + 日志实现 (Implementation) 解耦模式。Spring Boot 默认使用 SLF4J + Logback, 通过配置文件自定义日志级别 (TRACE → DEBUG → INFO → WARN → ERROR) 和输出方式。 [p.29–32, p.37]
- **事务管理** [p.33–36, p.37]: ACID 四属性——原子性 (Atomicity)、一致性 (Consistency)、隔离性 (Isolation)、持久性 (Durability)。编程式事务 (灵活但冗余) vs 声明式事务 (@Transactional, 基于 AOP, 无侵入, 项目主流)。 [p.33–36, p.37]
- **Actuator 运维监控** [p.9, p.19, p.37]: 通过 spring-boot-starter-actuator 提供 health、metrics、env、loggers、mappings 等生产就绪端点。 [p.9, p.19, p.37]

6 考试重点速查表 (Exam Quick Reference)

主题 (Topic)	关键词 (Key-words)	核心考点 (Key Points)
Spring Boot 定义	脚手架、约定优于配置	简化 Spring 使用方式, 非功能替代。 [p.6]
自动配置入口	@SpringBootApplication	@SpringBootConfiguration + @EnableAutoConfiguration + @ComponentScan。 [p.20]
条件注解	@ConditionalOnXXX	条件判断决定配置是否生效。 [p.20–21]
Back off 机制	@ConditionalOnMissingBean	用户自定义 Bean 优先, 自动配置失效。 [p.21]
Starter	spring-boot-starter-*	统一依赖管理 (版本仲裁), 继承 parent POM。 [p.17]
配置文件	.properties / .yaml	优先级: 命令行 > 环境变量 > 外部 > 内部。 [p.22, p.24]
多环境	@Profile / spring.profiles.active	application-{profile}.yaml, 三种激活方式。 [p.24]
内嵌容器	嵌入式 Tomcat	运行时动态创建, java -jar 直接运行。 [p.25]
日志门面 + 实现	SLF4J + Logback	门面模式, 默认日志方案, 通过配置自定义。 [p.30, p.32]
声明式事务	@Transactional	基于 AOP 代理, 无侵入, 项目主流方式。 [p.35]
事务 ACID	原子性/一致性/隔离性/持久性	事务四大基本属性, 保证数据完整性。 [p.33]
日志级别顺序	TRACE→DEBUG→INFO→WARN→ERROR	从低到高级别, 高级别包含低级别日志。 [p.32]
自动配置原理	条件判断 + 默认值	扫描.imports 文件, 条件注解筛选, 按需装配。 [p.20–21]
监控工具	Actuator	/health, /metrics, /env, /beans, /loggers 等端点。 [p.19]

端口配置	server.port	默认 8080，设为 0 随机端口，设为 -1 关闭 HTTP。[p.26]
------	-------------	--